# 王道 2024 考研 408 真题参考答案

重要通知　：王道以下课程名额已不多，有兴趣的同学请尽早扫码咨询，锁定名额。

**王道寒假短期班**—累计 45 天集训。适用于希望在复试前参与完整的项目开发、提升复试竞争力的同学。

**王道长期班**—累计 4 个月集训。适用于希望在研究生入学前提升工程能力的同学，也适合集训后想直接直接找工作就业的同学。




> 说明：
>
> 1. 该文档由王道热心学员+王道团队共同整理，感谢为我们提供题目信息的各位同学。各位友商若要参考该资料，请注明出处。
> 2. 参考答案由王道团队编写，其中难免会有错误，如果大家发现参考答案的问题，可以联系王道工作人员反馈。
> 3. 该文档近期可能还会有更新。如有错误或有补充，会实时同步到这里，大家可以收藏文档查看链接。

## 选择题：

### 数据结构：

1. 一个带头结点的链表 L，指针 p 指向中间的一个链表结点（不是第一个和最后一个结点）。q=p->next, p->next=q->next, q->next=L->next, L->next=q。这段代码的功能是（　）。

C. 将 p 结点移动到表头

D. 将 q 结点移动到表头

2. 表达式 x+y*(z-u)/v 的等价后缀是（　）。

A. xyzu-*v/+

B. xuzu-v*+

C. +x/*y-zuv

D. +x*y/-zuv

3. p、q、v 都是二叉树 T 中的结点，二叉树 T 的中序遍历位…p, v, q, …，其中 v 有两个孩子结点，则（ ）。

A. p 没有右孩子，q 没有左孩子

B. p 没有右孩子，q 有左孩子

C. p 没有右孩子，q 没有左孩子

D. p 没有右孩子，q 有左孩子

4. 邻接多重表中顶点 B 和 D 的度分别是多少？

B. 2、4

5. 不适用于折半查找的是（ ）。

Ⅰ有序链表

Ⅱ无序数组

Ⅲ有序静态链表

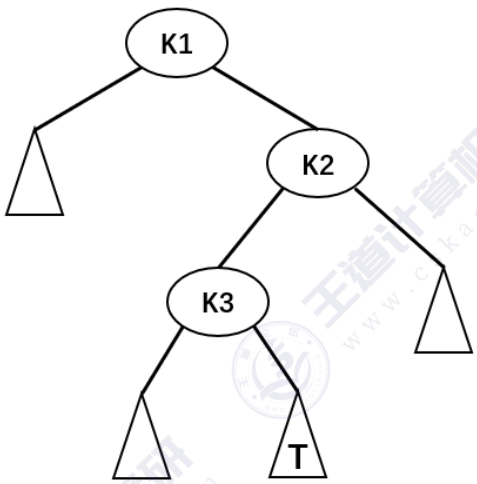
Ⅳ无序静态链表

Ⅰ、Ⅱ、Ⅲ、Ⅳ

6. 修正后的 KMP 算法进行模式匹配，模式串 S=“aabaab”，主串中某字符失配时，S 右滑最长距离为（ ）。

A. 5　　B. 4　　C. 3　　D. 2

7.二叉搜索树中 K1、K2、K3 是结点的关键字、三角形表示子树。则子树 T 中任意结点保存的关键字 x 满足（ ）。



A. X<K1

B. X>K2

C. K1<x<K3

D. K3<x<K2

8. 使用快速排序算法对含 N 个元素的数组 M 进行排序，若第一趟排序将除枢轴外的 N-1 个元素划分为 P 和 Q 两个部分，则下列叙述中，正确的是（ ）。

A. P 和 Q 块间有序

B. P 和 Q 均块内有序

C. P 和 Q 的元素个数大致相等

D. P 和 Q 中均不存在相等的元素

9. 大根堆初始序列为 28, 22, 20, 19, 8, 12, 15, 5，对该堆进行两次删除操作后，得到的新堆是（ ）。

A. 20, 19, 15, 12, 8, 5

B. 20, 19, 15, 5, 8, 12

C. 20, 19, 12, 15, 8, 5

D. 20, 19, 8, 12, 15, 5

10. 初始有三个升序序列(3, 5)、(7, 9)、(6)，采用二路归并，则关键字比对次数时（ ）。

A. 3　　B. 4　　C. 5　　D. 6

11. 外部排序使用败者树进行升序归并，记录“冠军”节点保存的是（ ）。

A. 最大关键字

B. 最大关键字所在的归并段号

C. 最小关键字

D. 最小关键字所在的归并段号

## 组成原理：

12. C 语言代码如下：

```
int i = 32777;
short si =i;
int j=si;
```

执行后 j 的真值为（ ）。

A. -32777　　B. -32759　　C. 32759　　D. 32777

13. 汇编指令序列->伪指令，CPU 能理解并直接执行的是

Ⅰ伪指令　Ⅱ微指令　Ⅲ机器指令　Ⅳ汇编指令

A. 仅 Ⅰ和 Ⅳ　　B. 仅 Ⅱ和 Ⅲ　　C. 仅 Ⅲ和 Ⅳ　　D. 仅 Ⅰ、Ⅲ和 Ⅳ

14. 整型参数α、β的取值范围分别为$-2^{-20}$~$2^{20}$、$-2^{-40}$~$2^{40}$，要求在保证数据精度的基础上提高运算速度，α和β最适宜采用（ ）。

A. 32 位整数、32 位整数　　B. 单精度浮点数、单精度浮点数

C. 32 位整数、双精度浮点数　　D. 单精度浮点数、双精度浮点数

15. 整数乘法运算中，错误的是（ ）。

A. 用阵列乘法器实现乘运算可以在一个时钟周期完成

B. 用 ALU 和移位器实现的乘运算无法在一个时钟周期内完成

C. 变量与常数的乘运算可编译优化为若干条移位及加减运算指令

D. 两个变量的乘运算无法编译为移位及加法等指令的循环实现

16. 关于存储器存储结构，说法错误的是（ ）。

A. Cache-主存层次的交换单位为主存块，主存-外存冲刺的交换单位为页

B. Cache-主存层次替换算法由硬件实现，主存-外存层次由软件实现

C. Cache-主存层次可采用回写法写策略，主存-外存层次通常采用回写法

D. Cache-主存层次可采用直接映射，主存-外存层次通常采用直接映射

17. 虚拟地址 VA 是 32 位，主存地址为 30 位，页大小为 1KB，TLB 有 32 个表项，采用 4 路组相联映射，则 TLB 标记字段位数至少是（ ）。

A. 17　　B. 18　　C. 19　　D. 20

18. 不是在 MMU 地址转换过程中检测的是（ ）。

A. 访问越权　　B. Cache 缺失　　C. 页面缺失　　D. TLB 缺失

19. 5 段流水线 RISC 说法错误的是（ ）。

C. 所有数据冒险都可以通过加入转发（旁路）电路解决

20. 存储器总线的时钟频率为 420MHz，总线宽度为 64 位，每个时钟周期传送 2 次数据，支持突发传输，最多传 8 次，第一个时钟传地址和读写命令，从第 4~7 个周期始终连续传 8 次。总线带宽最大传输速率为（ ）。

A. 3.84GB/s　　B. 6.72GB/s　　C. 30.72GB/s　　D. 53.76GB/s

21. 关于中断 I/O 方式，错误的是（ ）。

A. 中断屏蔽字决定中断响应顺序

C. 保存通用寄存器和设置新中断屏蔽字由软件实现

D. 单重中断方式下，中断处理时 CPU 处于关中断状态

22. DMA 方式中，DMA 控制器控制的数据传输通路位于（ ）。

C. 设备接口和主存之间　　D. 设备接口和 DMA 控制器之间

## 操作系统：

23. 下面关于中断和异常的说法中，错误的是（ ）。

A. 中断或异常发生时，CPU 处于内核态

24. 终止进程时，不一定执行的是（ ）。

A. 终止子进程

25. 支持页式存储管理的系统，进程切换时 OS 要执行（ ）。

Ⅰ更新 PC 值　　Ⅱ更新栈基址寄存器值（ebp）

Ⅲ更新页表基址寄存器值

D. Ⅰ、Ⅱ、Ⅲ

26. 文件系统需要额外的外存空间记录空闲块的位置，占用外存空间大小与当前空闲块数量无关的是（ ）。

A. 位图法

27. 回收分区时，仅合并大小相等的空闲分区的算法是（ ）。

A. 伙伴算法　　B. 最佳适应算法　　C. 最坏适应算法　　D. 首次适应算法

28. 进程 P 有一个线程 T，打开文件后获得 fd，再创建线程 Ta、Tb，则线程 Ta、Tb 可共享的资源是（ ）。

Ⅰ进程 P 的地址空间　　Ⅱ线程 T 的栈　　Ⅲfd

B. 仅 Ⅰ Ⅲ

29. 包含文件按名查找功能的系统调用是（ ）。

A. open()　　B. read()　　C. write()　　D. close()

30. RR 调度，时间片为 5ms，有 10 个进程，初始状态均处于就绪队列，执行结束前仅处于执行态或就绪态，队尾进程 P 所需 CPU 时间最短，为 25ms，不考虑系统开销，则 P 的周转时间为（ ）。

C. 250ms

31. 键盘中断服务例程执行结束时，所输入的数据存放位置是（ ）。

A. 用户缓冲区　　B. CPU 的通用寄存器

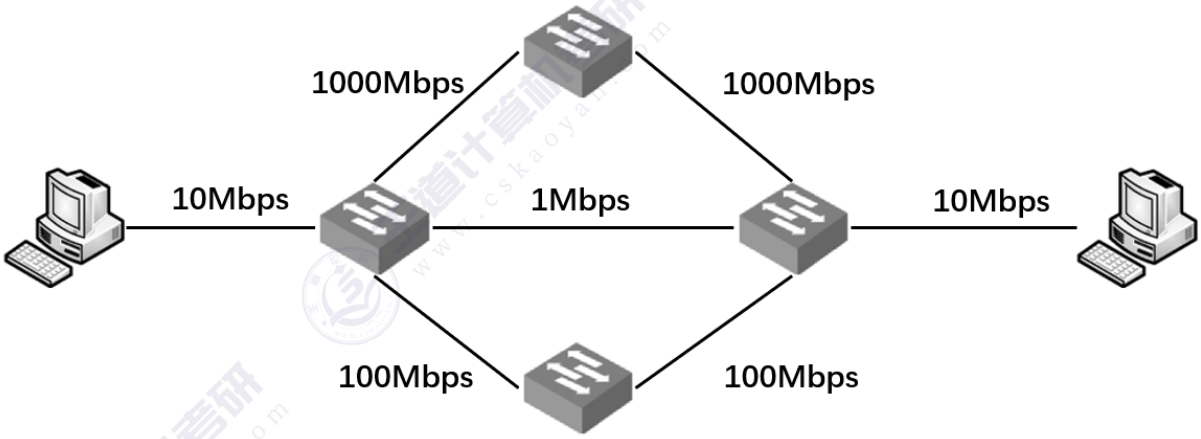
C. 内核缓冲区　　D. 键盘控制器的数据缓冲区

32. 磁盘调度算法是 CSCAN，磁道号是 0~399，完成 200 号磁道请求后，磁头向磁道号变小的方向移动，此时有 7 个 请求，300, 120, 110, 0, 160, 210, 399。完成上述访问请求后，磁头移动距离是（ ）。

A. 599　　B. 619　　C. 788　　D. 799

## 计算机网络：

33. 若分组交换网络及每段链路的带宽如下图，则 H1 到 H2 的最大吞吐量约为（ ）。



A. 1Mbps　　B. 10Mbps　　C. 100Mbps　　D. 1000Mbps

34. 在下列二进制数字调制方法中，需要 2 个不同频率载波的是（　）。

A. ASP　　B. PSK　　C. FSK　　D. DPSK

35.

36. 802.11 采用 CSMA/CA 协议进行通讯，SIFS 为 28μs，DIFS 为 120μs，RTS、CTS、ACK 的传播时延分别为 3μs、2μs、2μs，主机 A 要向 AP 发送 1998B 的数据，数据传输速率为 54Mbps，求隐蔽站 B 接收到 CTS 之后至少要将 NAV 值设置为多少μs？（　）。

A. 326

B. 354

C. 385

D. 5xx

> 其他站收到 CTS 帧后，根据 CTS 帧中的持续时间修改自己的网络分配向量 NAV(Network Allocation Vector)。
>
> NAV 的时间就是其他结点推迟访问的时间，约等于 SIFS 时间+数据发送时间+SIFS 时间+ACK 帧传播时延。
>
> 见谢希仁《计算机网络（第 7 版）》p402

### 2. 对信道进行预约

为了更好地解决隐蔽站带来的碰撞问题，802.11 允许要发送数据的站对信道进行**预约**。具体的做法是这样的。如图 9-8 所示，A 在向 B 发送数据帧之前，先发送一个短的控制帧，叫做**请求发送** RTS（Request To Send），它包括源地址、目的地址和这次通信（包括相应的确认帧）所需的持续时间。当然，A 在发送 RTS 帧之前，必须先监听信道。若信道空闲，则等待一段时间 DIFS 后，才能够发送 RTS 帧。若 B 正确收到 A 发来的 RTS 帧，且媒体空闲，则等待一段时间 SIFS 后，就向 A 发送一个叫做**允许发送** CTS (Clear To Send) 的控制帧，它也包括这次通信所需的持续时间。A 收到 CTS 帧后，再等待一段时间 SIFS 后，就可发送数据帧。若 B 正确收到了 A 发来的数据帧，在等待时间 SIFS 后，就向 A 发送确认帧 ACK。

这里要说明一下。根据 802.11 协议的规定，A 在 RTS 帧中所填写的所需占用信道的持续时间，是从 RTS 帧发送完毕后，到 B 最后发送完确认 ACK 为止的时间。从图 9-8 可以看出，这段时间就是[SIFS + CTS + SIFS + 数据帧 + SIFS + ACK]这些时间段之和。这个时间也就是从 RTS 帧发完后，信道仍被 A 和 B 的通信占用的时间。而 B 在 CTS 帧中所填写的所需占用信道的持续时间，是从 CTS 帧发送完毕后，到 B 最后发送完确认 ACK 为止的时间。从图 9-8 可以看出，这就是[SIFS + 数据帧 + SIFS + ACK]这些时间段之和。这个时间也就是从 CTS 帧发完后，信道仍被 A 和 B 的通信占用的时间。

上述的这两个持续时间，实际上分别是 A 和 B 的网络分配向量 NAV。其他站注意到 NAV 后，就不会在这段时间发送数据了。

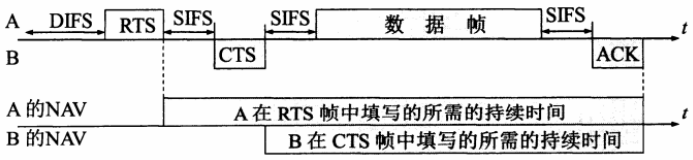



图 9-8　发送 RTS 帧和 CTS 帧对信道进行预约

使用 RTS 帧和 CTS 帧会使整个网络的通信效率有所下降。但由于这两种控制帧都很短，其长度分别为 20 字节和 14 字节，与数据帧（最长可达 2346 字节）相比开销不算大。相反，若不使用这种控制帧，则一旦发生碰撞而导致数据帧重发，浪费的时间就更多了。虽然如此，协议还是设有三种情况供用户选择。



谢希仁《计算机网络（第 8 版）》p418

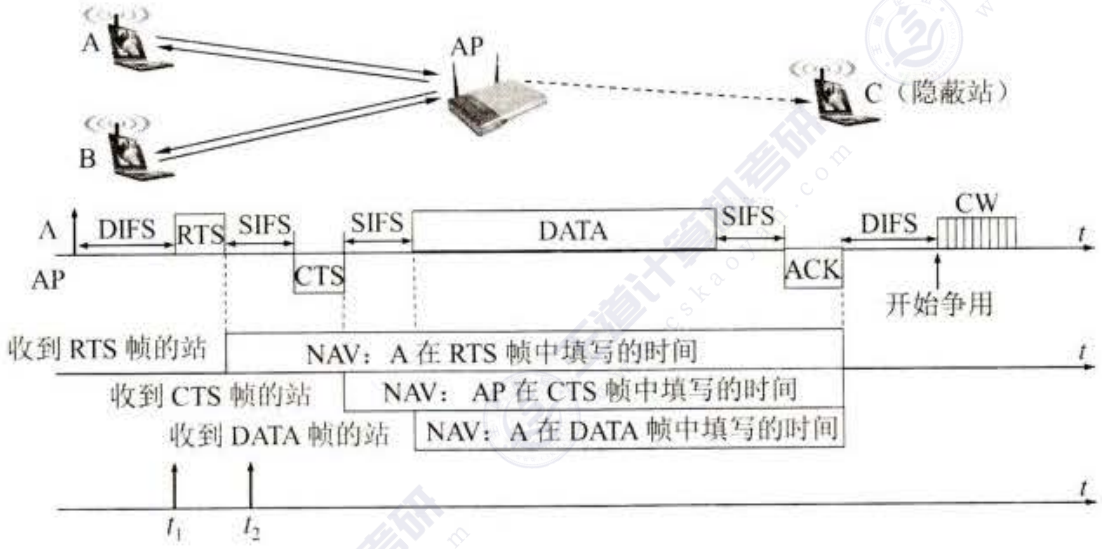



图 9-10　发送 RTS 帧和 CTS 帧对信道进行预约

37. 主机甲通过选择重传（SR）滑动窗口协议向主机乙发送帧的部分过程如下图所示。F 为数据帧，ACKx 为确认帧，X 是帧序号。乙只对正确接收的数据帧进行独立确认。发送窗口与接收窗口大小相同且均为最大值。甲在 t1 时刻和 t2 时刻发送的数据帧分别是（　）。

A. F1、F3　　B. F1、F4　　C. F3、F1　　D. F4、F1

38. TCP 连接和断开，时间是（　）。

D. 60.04

39. UDP 校验和计算

> 具体题目数字不确定，有记得的同学可以联系我们。
>
> 校验和计算根据题目给出的两个 0/1 序列相加（中间需要进位），得到结果（注意：最高位如果产生进位需要在末尾加 1）。将结果再取反即是最终校验和。

40. 若浏览器不支持并行 TCP 连接，使用非持久的 HTTP/1.0 协议请求浏览 1 个 web 页，该页中引用同一个网站上 7 个小图像文件，则从浏览器传输 web 页请求建立 TCP 连接开始后，到接收完所有内容为止。所需要的往返时间 RTT 数至少是（　）。

A. 4　　B. 9　　C. 14　　D. 16

# 大题：

## 数据结构：

41.（13 分）有向图 G 采 2023 年 10 月 26 日，神舟十七号载人飞船发射活动取得圆满成功，这再彰显了我国在航天事业取得辉煌成就。载人航天工程是包含众多子工程的复杂系统工程，为确保工程的有序开展，需要明确各子工程的前导子工程，来协调各子工程的实施。该问题可以简化、抽象为有向图的拓扑序列问题。已知有向图 M 采用邻接矩阵存储，结构定义如下:

```
typedef struct                          // 图的类型定义
{
    int  numVertices, numEdges;         // 图的顶点数和有向边数
    char     VerticesList[ MAXV ];      // 顶点表，MAXV 为已定义常量
    int  Edge[ MAXV ][ MAXV ];          // 邻接矩阵
} MGraph;
```

实现算法 int uniquely（MGraph G），判断 G 是否有唯一的拓扑序列，如果存在则返回 1，否则返回 0。

（1）描述算法思想；（4 分）

（2）用 C 或 C++语言实现算法。（9 分）

【参考答案 1】

（1）①统计每个顶点的入度。

②循环 numVertices 趟，每趟执行下述过程：

a. 统计入度为 0 的顶点数 count。如果 count 不为 1 则说明拓扑序列不唯一或者不存在，函数返回 0。

b. 对于入度不为 0 的顶点 u，将它的入度设为-1。如果 u->j 之间存在边，则顶点 j 的入度-1。

③如果循环执行完，则说明拓扑序列唯一，函数返回 1。

```
int uniquely(MGraph G){
    int inDegree[MAXV]={0};
    // 计算每个顶点的入度
    for(int i = 0; i < G.numVertices; ++i) {
        for(int j = 0; j < G.numVertices; ++j) {
            if(G.Edge[i][j] > 0) {
                ++inDegree[j];
            }
        }
    }
    int count, u;
    for(int i = 0; i < G.numVertices; ++i) { //拓扑序列第 i 个元素
        count = 0;                          //计数器清零
        for (int j = 0; j < G.numVertices; ++j) {
            if (inDegree[j] == 0) {
                u = j;
                count++;                    //统计入度为 0 的顶点数
            }
        }
        //入度为 0 的顶点数>1，则拓扑序列不唯一；如果=0，则拓扑序列不存在
        if (count != 1) return 0;

        inDegree[u] = -1;   //u 为本轮拓扑排序输出的顶点
        for (int j = 0; j < G.numVertices; ++j) {
            if (G.Edge[u][j] != 0) {    //存在有向边 u->j
                --inDegree[j];          //顶点 j 的入度-1
```

【参考答案 2】

（1）①统计每个顶点的入度，将入度为 0 的顶点入队。

②如果队列不为空则循环执行：

a. 如果队列中元素个数大于 1 则拓扑序列不唯一，函数返回 0。

b. 出队队首元素 u，如果存在边 u->i，则将顶点 i 的入度-1，如果它的入度变为 0 则将 i 入队。

③如果所有顶点都已入队过，说明拓扑序列唯一，函数返回 1。

如果还有顶点没有入队过，说明不存在拓扑序列，函数返回 0。

（2）

```
int uniquely(MGraph G){
    int queue[MAXV],inDegree[MAXV]={0};
    int front=0, rear=0;
    // 计算每个顶点的入度
    for(int i = 0; i < G.numVertices; ++i) {
        for(int j = 0; j < G.numVertices; ++j) {
            if(G.Edge[i][j] > 0) {
                ++inDegree[j];
            }
        }
    }
    // 入度为 0 的顶点入队
    for (int i = 0; i < G.numVertices; i++) {
        if (inDegree[i] == 0) {
            queue[rear++] = i; // 入队
        }
    }
    while (front < rear) {                  //队列非空
        if (rear - front > 1) return 0;       //拓扑序列不唯一
        int u = queue[front++];               //出队
        // 减少与它相邻顶点的入度
        for (int i = 0; i < G.numVertices; i++) {
            if (G.Edge[u][i] != 0) {
                if (--inDegree[i] == 0) {
                    queue[rear++] = i;        //入度为 0 的顶点入队
                }
```

42.（10 分）将关键字 20, 3, 11, 18, 9, 14, 7 依次存储到长度为 11 的散列表 HT，散列函数为 H（key）=（key×3）%11，H0 为初始散列地址，H1、H2、H3... Hk 分别为第 1 次冲突、第 2 次冲突、第 3 次冲突...第 k 次冲突时探测的地址。Hk=（H0+k^2）%11

（1）画出 HT，算装填因子。（6 分）

（2）查找关键字 14 时的关键字比较序列。（2 分）

（3）查找关键字 8 失败时的哈希地址。（2 分）

【参考答案】

（1）

| 散列地址 | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 关键字 | 11 | | 14 | 7 | | 20 | 9 | | | 3 | 18 |
| 比较次数 | 1 | | 3 | 2 | | 1 | 2 | | | 1 | 1 |

装填因子=7/11

> 注：表格中比较次数不需要在答题时给出，此处为了方便大家理解
>
> ①H(20) = 5，关键字 20 装入散列地址 5
>
> ②H(3) = 9，关键字 3 装入散列地址 9
>
> ③H(11) = 0，关键字 11 装入散列地址 0
>
> ④H(18) = 10，关键字 18 装入散列地址 10
>
> ⑤H(9) = 5，冲突；H1(9) = (5+1)%11 = 6，关键字 9 装入散列地址 6
>
> ⑥H(14) = 9，冲突；H1(14) = (9+1)%11 = 10，冲突；H2(14) = (9+4)%11 = 2，关键字 14 装入散列地址 2

（2）3、18、14

> H(14) = 9，和关键字 3 比较，不命中；H1(14) = (9+1)%11 = 10，和关键字 18 比较，不命中；H2(14) = (9+4)%11 = 2，和关键字 14 比较，命中。

（3）哈希地址为 7

> H(8) = 2，和关键字 14 比较，不命中；H1(8) = (2+1)%11 = 3，和关键字 7 比较，不命中；H2(8) = (2+4)%11 = 6，和关键字 9 比较，不命中；H3(8) = (2+9)%11 = 0，和关键字 11 比较，不命中；H4(8) = (2+16)%11 = 7，是空位，查找失败。

## 计算机组成原理：

43.（13 分）计算机 M，字长 32 位，按字节编址，采用 32 位定长指令字。add、slli、lw 三条指令的机器码、功能说明如下：

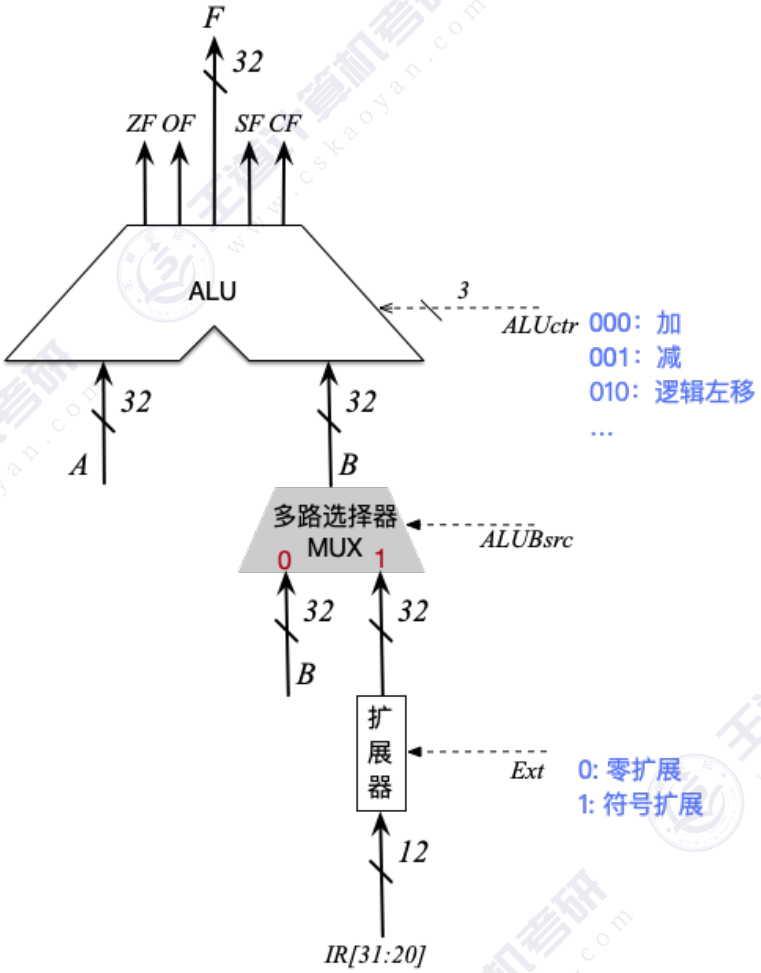
| 位数 | 31 25 | 24 20 | 19 15 | 14 12 | 11 7 | 6 0 |
|---|---|---|---|---|---|---|
| add 指令 | 0000000 | rs2 | rs1 | 000 | rd | 0110011 |
| slli 指令 | 0000000 | shemt | rs1 | 010 | rd | 0010011 |
| lw 指令 | imm | | rs1 | 010 | rd | 0000011 |

第一条是 add 指令 R[rd] <- R[rs1]+R[rs2]

第二条是 slli 指令，R[rd] <- R[rs1]<<shemt

第三条是 lw 指令，R[rd] <- M[R[rs1]+imm]

imm 是补码，shamt 表示移位位数，输入端 A 的数据来自寄存器 rs1，输入端 B 的数据来自 rs2。电路图如下：




1. M 最多有几个寄存器，为什么 shemt 占 5 位。 （2 分）
2. add 指令，ALUBsrc 为？若 rs1 为 8765 4321H，rs2 为 9876 5432H，则 add 执行后，F,OF,CF 为多少？若 add 处理的是无符号数，则应该根据哪个标志位判断溢出?（5 分）
3. 执行 slli 时，控制信号 Ext 可以是 0 也可以是 1，为什么？（2 分）
4. 执行 lw 指令时，Ext，ALUctr 为多少？（2 分）
5. 若指令的机器码是 A040 A103H，则一定是 lw 指令，为什么？若执行该指令时，R[01H]=FFFF A2D0H,则读取数据的存储地址是多少？（2 分）

【参考答案】

1. 最多有 32 个寄存器。（1 分）

   因为计算机 M 的机器字长是 32 位，因此 slli 指令处理的操作数位数是 32 位，而 32 位操作数最多只需左移 31 位即可。若左移 32 位，会导致 32bit 操作数的所有有效信息位丢失。因此，shemt 表示左移位数，5bit 可表示范围是 0~31，可以满足需求。（1 分）

   > 注：第 1 题第 2 小问可以回答的更简洁，只需要提到“因为该计算机的机器字长（或寄存器）是 32 位”即可得分

2. ALUBrc=0；（1 分）

   F=1FDB 9753H；（1 分）

   OF=1；（1 分）

   CF=1；（1 分）

   根据 CF 标志位判断溢出。（1 分）

3. slli 指令的高 12 位（即 IR[31:20]）的最高位为 0，因此无论进行零扩展还是符号扩展，都是在高位补 0，效果等价，因此 Ext 可以是 0 也可以是 1。（2 分）

4. Ext=1；（1 分）

   ALUctr=000。（1 分）

5. 因为机器码 A040A103H=101000000100 00001 010 00010 0000011B， 14~12 位=010，6~0 位=0000011。（1 分）

   指令的高 12 位=A04H，经过符号扩展后，得到 32 位机器码=FFFF FA04H，读取数据的存储地址 = FFFF A2D0H + FFFF FA04H = FFFF 9CD4H。（1 分，答题时可不写过程）

44.（10 分）承接 43 题条件，C 语言程序“sum+=a[i]”，对应的指令如下：

```
slli r4, r2, 2        //R[r4] <- R[r2]<<2
add r4, r3, r4        //R[r4] <- R[r3]+R[r4]
lw   r5, 0(r4)        //R[r5] <- M[R[r4]+0]
add  r1, r1, r5       // R[r1] <- R[r1]+R[r5]
```

i,sum,数组 a 都为 int 型，r1~r5 的寄存器编号为 01H~05H

1. a 的首地址、变量 i、变量 sum 存储的寄存器编号是？（3 分）
2. 该系统采用小端方式，页式存储，页大小为 4KB。执行第 1 条指令时，i=5，r1=0000 1332H，r3=0013 DFF0H，存储单元内容如下所示，执行 sum+=a[i] 后，a[i] 的地址为？a[i] 和 sum 的机器数为？a[i] 所在页的页号为？此次执行中，数组 a 至少存放在几页中？（5 分）

| 地址 | | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|---|---|
| 0013 | DFF0 | FF | FF | FF | 7C | 70 | FE | FF | FF |

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| 0013 | DFF8 | 00 | 00 | 00 | 0C | 3C | 02 | 01 | FF |
| 0013 | E000 | F0 | F1 | 00 | 00 | DC | EC | FF | FF |
| 0013 | E008 | FF | FF | 01 | 02 | 00 | 00 | 01 | 02 |

3. slli r4，r2，2 的机器码为？若 a 改为 short 型，则 slli 指令的汇编形式应该是？（2 分）

【参考答案】

1. 数组 a 的首地址寄存器编号=03H。（1 分）
   变量 i 的首地址寄存器编号=02H。（1 分）
   变量 sum 的首地址寄存器编号=01H。（1 分）
2. a[i]的地址=0013 E004H。（1 分）
   a[i]的机器数=FFFF ECDCH。（1 分）
   sum 的机器数=0000 000EH。（1 分）
   a[i]所在页页号=0013EH。（1 分）
   数组 a 至少存放在 2 页中。（1 分）
3. 机器码=0000000 00010 00010 010 00100 0010011= 0021 2213H。（1 分，可不写过程）
   若 a 改为 short 型，slli 指令的汇编形式应该是 slli r4, r2, 1。（1 分）

## 操作系统：

45. （7 分）某计算机采用虚拟页式存储管理，虚拟地址、物理地址为 32 位，页表项大小为 4B，页面大小为 4MB。虚拟地址结构如下：

| 页号（10 位） | 页内偏移量（22 位） |
|---|---|

进程 P 的页表起始虚拟地址为 B8C0 0000H，页表被装载到物理地址为 6540 0000H 开始的连续空间。

（1）进程 P 访问虚拟地址 1234 5678H 时发生缺页，经缺页异常处理和 MMU 地址转换之后，得到的物理地址=BAB4 5678H，此次缺页异常处理过程中，需要为所缺页面分配页框，并更新相应的页表项，则该页表项的虚拟地址、物理地址分别是？该页表项的页框号更新后的值是什么？（3 分）

（2）进程 P 的页表所在页的页号是多少？该页对应的页表项的虚拟地址是多少？该页表项中的页框号是多少？（4 分）

【参考答案】

（1）该页表项的虚拟地址=B8C0 000H + 048H * 4 = B8C0 0120H。（1 分，可不写过程）

该页表项的物理地址=6540 0000H + 048H * 4 = 6540 0120H。（1 分，可不写过程）

页框号等于物理地址 BAB4 5678H 的前 10 位，即 1011101010 B= 2EAH。（1 分，可不写过程）

> 注 1：虚拟地址 1234 5678H 的页号=0001001000B = 048H。页表项大小为 4B。
>
> 注 2:  将 048H 展开为二进制，并左移两位，即可等效“乘以 4”的效果。048H*4=120H。

（2）进程 P 的页表所在页的页号等于 B8C0 0000H 的前 10 位，即 1011100011B=2E3H。（1 分，可不写过程）

该页对应的页表项的虚拟地址=B8C0 0000H + 2E3H * 4 =B8C0 0B8CH。（1 分，可不写过程）

该页表项中的页框号等于物理地址 6540 0000H 的前 10 位，即 0110 0101 01B = 195H。（预计 2 分，因为这个问题比较绕，难度更高）

46.（8 分）一个网络系统，缓冲区 B 可以存放一个数据分组。有 C1、C2、C3 三种操作。

C1：把一个数据分组写入 B（B 空时才能执行）

C2：从 B 中读一个分组（B 非空时才能执行）

C3：对 B 中的数据分组进行修改（B 非空时才能执行）

1. 若进程 P1、P2 都要执行 C1，实现 C1 的代码是否为临界区？为什么？（2 分）
2. 假设 B 初始为空，P1 执行 C1 一次，P2 执行 C2 一次。请定义尽可能少的信号量，用 wait（），signal（）操作描述 P1、P2 之间的同步互斥关系，说明信号量作用和初值。（3 分）
3. 假设 B 初始非空，P1、P2 各执行 C3 一次，请定义尽可能少的信号量，用 wait（），signal（）操作描述 P1、P2 之间的同步互斥关系，说明信号量作用和初值。（3 分）

【参考答案】

1. 实现 C1 的代码是临界区。（1 分）

   因为缓冲区 B 属于临界资源，各进程应互斥访问。（1 分，回答方式不唯一）
2. (3 分)，参考代码如下。

```
semaphore full=0;   //同步信号量，表示缓冲区 B 内的数据分组数量

P1(){   //P1 进程
   执行 C1；        //将一个数据分组写入缓冲区 B
   signal(full);   //缓冲区内数据分组数量+1
}

P2(){   //P2 进程
   wait(full);   //若缓冲区 B 为空，进程 P2 阻塞等待
   执行 C2;      //从缓冲区 B 读出一个数据分组
}
```

> 第 2 小问 很多同学会再定义一个互斥信号量，以保证对临界资源的互斥访问。然而这题的要求比较特殊，让我们用尽可能少的信号量。由于缓冲区 B 初始为空，且最大容量仅为 1，因此只需一个同步信号量即可确保 P1 先写、P2 后读。本题题干仅要求我们描述 P1、P2 之间的同步互斥关系，因此不需要考虑还包含其他进程的情况，不用把问题想的太复杂。
>
> 如果你定义了多个信号量，只要能正确实现同步关系，预计最多扣 1 分。请大家别慌！

3. (3 分)参考代码如下。题干中明确表示缓冲区 B 初始非空，因此仅需使用一个互斥信号量即可。

```
semaphore mutex=1;   //互斥信号量

P1(){   //P1 进程
   wait(mutex);   //互斥访问临界区
   执行 C3；        //修改据缓冲区 B 中的数据
   signal(mutex);   //释放临界区
}

P2(){   //P2 进程
   wait(mutex);   //互斥访问临界区
   执行 C3；        //修改据缓冲区 B 中的数据
   signal(mutex);   //释放临界区
}
```

注：咸鱼猜测，用 PV 操作答题不应该被扣分，除非改卷老师不讲武德。因为 wait、signal 操作就是 P、V 操作。进程间同步互斥问题，最初由迪杰斯特拉开始研究，这位大佬是荷兰人，PV 操作的名字来自荷兰语。为了坚定各位的信心，下面摘录几本经典教材的文字。

以下这段文字摘录自《计算机操作系统（第四版）》—汤小丹。

### 2.4.3 信号量机制

1965 年，荷兰学者 Dijkstra 提出的信号量(Semaphores)机制是一种卓有成效的进程同步工具。在长期且广泛的应用中，信号量机制又得到了很大的发展，它从整型信号量经记录型信号量，进而发展为“信号量集”机制。现在，信号量机制已被广泛地应用于单处理机和多处理机系统以及计算机网络中。

**1. 整型信号量**

最初由 Dijkstra 把整型信号量定义为一个用于表示资源数目的整型量 S，它与一般整型量不同，除初始化外，仅能通过两个标准的原子操作(Atomic Operation) wait(S)和 signal(S)来访问。很长时间以来，这两个操作一直被分别称为 P、V 操作。wait 和 signal 操作可描述如下：

以下这段文字摘录自《操作系统概念（第 9 版）》—Abraham Silberschatz。这本书在世界范围内也非常具有权威性。

## 6.6 信号量

互斥锁，我们刚刚讨论过了，通常认为是最简单的同步工具。本节将会讨论一个更鲁棒 263
的工具，它的功能类似于互斥锁，但是它能提供更为高级的方法，以便进程能够同步活动。

一个**信号量**（semaphore）S 是个整型变量，它除了初始化外只能通过两个标准原子操作：wait() 和 signal() 来访问。操作 wait() 最初称为 P（荷兰语 proberen，测试）；操作

---



signal() 最初称为 V（荷兰语 verhogen，增加）。可按如下来定义 wait()：

**计算机网络：**

47.

【参考答案】

（1）OSPF

（2）TTL=16

（3）60s、90s

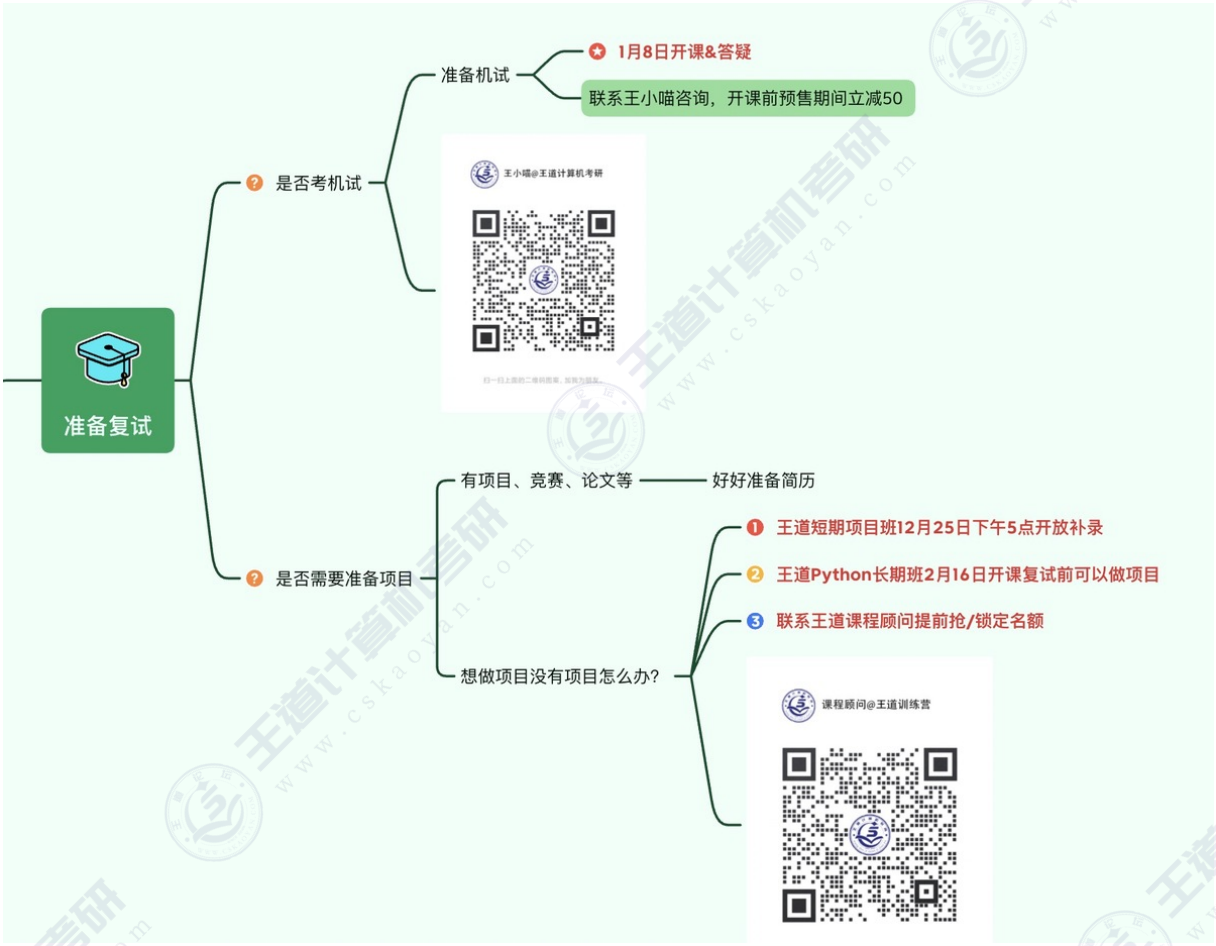
（4）更新报文

（5）$R_{11}$

$R_{13}$

# 初试结束后应该做什么？

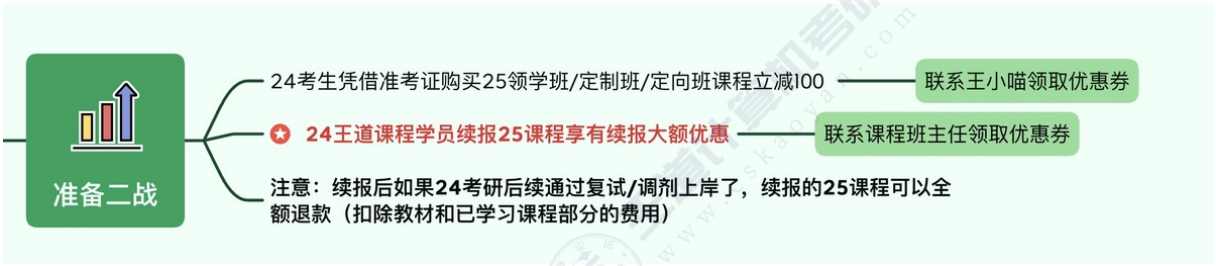
## 1. 准备复试


准备复试
是否考机试
准备机试
1月8日开课&答疑
联系王小喵咨询，开课前预售期间立减50
王小喵@王道计算机考研
是否需要准备项目
有项目、竞赛、论文等
好好准备简历
想做项目没有项目怎么办?
1 王道短期项目班12月25日下午5点开放补录
2 王道Python长期班2月16日开课复试前可以做项目
3 联系王道课程顾问提前抢/锁定名额
课程顾问@王道训练营

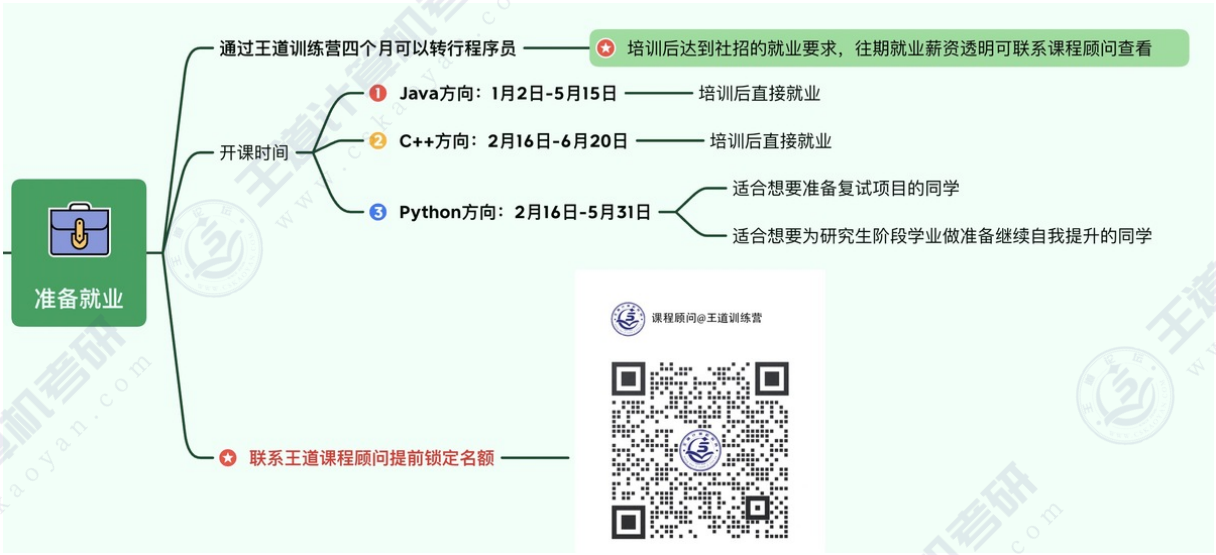
## 2. 准备调剂


准备调剂
进入复试调剂群
24王道考研调剂1群
群内分享内容
第一时间发布王道复试阶段系列直播通知
部分院校复试攻略
提供往年调剂信息参考
第一时间分享最新调剂信息

## 3. 准备二战




## 4. 准备就业




**复试调剂资讯和信息会第一时间发布在复试调剂群**，已经进群的同学无需重复进群：



24王道考研复试调...

群号: 310393506



扫一扫二维码，加入群聊

QQ